RapidMart - SQL Server Operations by Role

Demo data, role permissions and a SQL Server query for each of the 91 operations in the application | 26 September 2026

About this document

Every operation the RapidMart application offers is listed under the role that performs it, with the REST endpoint it maps to and a Microsoft SQL Server (T-SQL) query that does the same thing to **RapidMartDB**. Read operations are plain `SELECT` statements; write operations reproduce the application's rules and run inside a transaction with `SET XACT_ABORT ON`, and stop with a `THROW` (error 50001+) when a business rule is not met.

All queries use the demo data from `database/RapidMartDB_SSMS.sql` (run with `SeedDemoData = "1"`). They were executed one after another against a fresh demo install: every query ran without a SQL error, and the resulting data matched the application's behaviour.

Important: the application remains the source of truth. It hashes passwords (BCrypt), sends e-mail, renders bill PDFs and applies validation that SQL cannot. Use the read queries freely for reports and troubleshooting. Run the write queries on a test copy (for example `RapidMartTestDB`), not on live data, unless you mean to change it.

Demo accounts

Role	Email	Password	Portal	Demo name
MANAGER	manager@rapidmart.local	DemoManager21!	Staff	System Manager
CASHIER	cashier@rapidmart.local	DemoStaff21!	Staff	Nimali Cashier
INVENTORY_STAFF	inventory@rapidmart.local	DemoStaff21!	Staff	Kasun Inventory
DELIVERY_COORDINATOR	coordinator@rapidmart.local	DemoStaff21!	Staff	Dilani Coordinator
DELIVERY_STAFF	driver@rapidmart.local	DemoStaff21!	Staff	Ruwan Driver
DELIVERY_STAFF	driver2@rapidmart.local	DemoStaff21!	Staff	Chamara Driver
LOYALTY_OFFICER	loyalty@rapidmart.local	DemoStaff21!	Staff	Sanduni Loyalty
CUSTOMER	customer1@example.com	DemoCustomer21!	Customer	Amal Perera (Colombo)
CUSTOMER	customer2@example.com	DemoCustomer21!	Customer	Sithara Fernando (Kandy)

Staff sign in with an @rapidmart.local address; customer addresses must end in .com.

Demo orders (one open work item per portal)

Order	Customer	Status	Payment	Delivery	Total (LKR)	Used by
RM-DEMO-0001	customer1	DELIVERED	COD - PAID	DELIVERED (driver)	1,260.00	Bill BILL-DEMO-0001, refund PENDING 220.00, 12 points
RM-DEMO-0002	customer2	AWAITING_PAYMENT	EFT - PENDING	PENDING, unassigned	2,500.00	Cashier EFT verification
RM-DEMO-0003	customer1	CONFIRMED	CARD - PAID	PENDING, unassigned	2,760.00	Coordinator assignment; bill BILL-DEMO-0003
RM-DEMO-0004	customer2	CONFIRMED	COD - PENDING	ASSIGNED (driver)	1,880.00	Driver dispatch; bill generation
RM-DEMO-0005	customer1	OUT_FOR_DELIVERY	COD - PENDING	OUT_FOR_DELIVERY	1,080.00	Driver completion, COD settlement

Other demo data

Area	Contents
Catalogue	6 categories and 23 products with opening stock; Instant Coffee (BV-COF-004) is out of stock.
Inventory	PENDING restock request for 50 x BV-COF-004; opening INITIAL_STOCK movements.
Loyalty tiers	BRONZE 0 pts (1.00x), SILVER 500 (1.25x), GOLD 2000 (1.50x), PLATINUM 5000 (2.00x).
Offers	WELCOME5 5% (min 1,000, all), FRESH10 10% (min 3,000, Silver+), GOLD15 15% (min 5,000, Gold+).
Customers	customer2: SILVER, 640 points (two order earnings + a 40-point birthday adjustment); cart holds 2 x bananas, 1 x eggs, 6 x water. customer1: 12 points.

Area	Contents
Feedback	customer1 'Great delivery service' (PENDING); customer2 'Bananas were overripe' (RESPONDED, manager reply).
Manager	PENDING email change to store.manager@rapidmart.local; 4 audit entries; 3 notification-log rows.

Operations at a glance

Role	Operations	Count
Public visitor & authentication	PUB-1, PUB-2, PUB-3, PUB-4, PUB-5	5
Customer	CUS-1, CUS-2, CUS-3, CUS-4, CUS-5, CUS-6, CUS-7, CUS-8, CUS-9, CUS-10, CUS-11, CUS-12, CUS-13, CUS-14, CUS-15, CUS-16, CUS-17, CUS-18, CUS-19, CUS-20, CUS-21	21
Cashier (payments, refunds, COD, billing)	CSH-1, CSH-2, CSH-3, CSH-4, CSH-5, CSH-6, CSH-7, CSH-8, CSH-9, CSH-10, CSH-11, CSH-12, CSH-13, CSH-14, CSH-15, CSH-16	16
Inventory staff	INV-1, INV-2, INV-3, INV-4, INV-5, INV-6, INV-7, INV-8, INV-9, INV-10, INV-11, INV-12, INV-13, INV-14	14
Delivery coordinator	DCO-1, DCO-2, DCO-3, DCO-4, DCO-5, DCO-6, DCO-7, DCO-8	8
Delivery staff (driver)	DST-1, DST-2, DST-3, DST-4, DST-5	5
Loyalty officer	LOY-1, LOY-2, LOY-3, LOY-4, LOY-5, LOY-6, LOY-7, LOY-8	8
Manager	MGR-1, MGR-2, MGR-3, MGR-4, MGR-5, MGR-6, MGR-7, MGR-8, MGR-9, MGR-10, MGR-11, MGR-12, MGR-13, MGR-14	14
Total		91

Public visitor & authentication (5 operations)

Demo login: No login required

Anyone can browse the catalogue. Registration always creates a CUSTOMER. Passwords are BCrypt hashes produced by the application, so SQL can look accounts up but cannot verify or create a password; the SQL below reuses the demo hash for 'DemoCustomer21!'.

ID	Operation	Endpoint
PUB-1	Browse active categories	`GET /api/v1/categories/public`
PUB-2	Browse / search the product catalogue	`GET /api/v1/products/public?categoryId=&search=`
PUB-3	View one product	`GET /api/v1/products/public/{id}`
PUB-4	Register a customer account	`POST /api/v1/auth/register`
PUB-5	Sign in (account lookup)	`POST /api/v1/auth/login`

PUB-1 Browse active categories

`GET /api/v1/categories/public`

Active categories only.

```
SELECT id, name, description
FROM categories
WHERE active = 1
ORDER BY name;
```

PUB-2 Browse / search the product catalogue

`GET /api/v1/products/public?categoryId=&search=`

Active products with live stock, 12 per page sorted by name. Set @Search / @CategoryId to filter.

```
DECLARE @Search NVARCHAR(100) = N'rice', @CategoryId BIGINT = NULL, @Page INT = 0, @Size INT = 12;

SELECT p.id, p.sku, p.name, p.unit, p.price, c.name AS category, i.quantity AS in_stock
FROM products p
JOIN categories c ON c.id = p.category_id
LEFT JOIN inventory i ON i.product_id = p.id
WHERE p.active = 1
  AND (@CategoryId IS NULL OR p.category_id = @CategoryId)
  AND (@Search IS NULL OR LOWER(p.name) LIKE N'%' + LOWER(@Search) + N'%')
ORDER BY p.name
OFFSET @Page * @Size ROWS FETCH NEXT @Size ROWS ONLY;
```

PUB-3 View one product

`GET /api/v1/products/public/{id}`

Returns nothing for inactive products.

```
SELECT p.id, p.sku, p.name, p.description, p.price, p.unit, p.image_url, c.name AS category, i.quantity AS in_stock
FROM products p
JOIN categories c ON c.id = p.category_id
LEFT JOIN inventory i ON i.product_id = p.id
WHERE p.sku = 'DE-MLK-001' AND p.active = 1;
```

PUB-4 Register a customer account

`POST /api/v1/auth/register`

Customer emails must end in .com. Creates the user with the CUSTOMER role only.

```
SET XACT_ABORT ON;
DECLARE @Email VARCHAR(255) = 'nimal.silva@example.com';
IF @Email NOT LIKE '%_@_%.com' THROW 50001, 'Customer email must contain @ and end in .com.', 1;
IF EXISTS (SELECT 1 FROM app_users WHERE email = @Email) THROW 50002, 'Email is already registered.', 1;

BEGIN TRANSACTION;
INSERT INTO app_users (email, password_hash, full_name, phone, active)
VALUES (@Email, '$2a$10$CjBzY7Q0DSjj.4/gwY3aVOagL0WABIM.QaTc2QLRet.sRXtu4zaTq', 'Nimal Silva', '+94773333333', 1);
INSERT INTO user_roles (user_id, role) VALUES (SCOPE_IDENTITY(), 'CUSTOMER');
COMMIT;
```

PUB-5 Sign in (account lookup)

`POST /api/v1/auth/login`

The app compares the password hash in Java. This is the lookup it performs, plus the last-login stamp.

```
DECLARE @Email VARCHAR(255) = 'customer1@example.com';

SELECT u.id, u.email, u.full_name, u.active, u.password_hash,
       STRING_AGG(r.role, ',') AS roles
FROM app_users u
LEFT JOIN user_roles r ON r.user_id = u.id
WHERE u.email = @Email
GROUP BY u.id, u.email, u.full_name, u.active, u.password_hash;

UPDATE app_users SET last_login_at = SYSDATETIMEOFFSET() WHERE email = @Email AND active = 1;
```

Customer (21 operations)

Demo login: customer1@example.com, customer2@example.com / DemoCustomer21! (customer portal)

Customers manage their profile, addresses and cart, check out, track and cancel orders, request refunds, view loyalty points and offers, and send feedback. Demo: customer2 is a SILVER member (640 points) with 3 items in the cart; customer1 owns RM-DEMO-0001/0003/0005.

ID	Operation	Endpoint
CUS-1	View my profile	`GET /api/v1/customer/profile`
CUS-2	Update my profile	`PUT /api/v1/customer/profile`
CUS-3	List my delivery addresses	`GET /api/v1/customer/addresses`
CUS-4	Add a delivery address	`POST /api/v1/customer/addresses`
CUS-5	Edit a delivery address	`PUT /api/v1/customer/addresses/{id}`
CUS-6	Delete a delivery address (soft delete)	`DELETE /api/v1/customer/addresses/{id}`
CUS-7	View my cart	`GET /api/v1/customer/cart`
CUS-8	Add an item to my cart	`POST /api/v1/customer/cart/items`
CUS-9	Change a cart item quantity	`PUT /api/v1/customer/cart/items/{itemId}`
CUS-10	Remove an item from my cart	`DELETE /api/v1/customer/cart/items/{itemId}`
CUS-11	Check out the cart (place an order)	`POST /api/v1/customer/orders/checkout`
CUS-12	List my orders	`GET /api/v1/customer/orders`
CUS-13	View one order with its items	`GET /api/v1/customer/orders/{id}`
CUS-14	Cancel my order	`POST /api/v1/customer/orders/{id}/cancel`
CUS-15	Request a refund	`POST /api/v1/customer/refunds`
CUS-16	List my refund requests	`GET /api/v1/customer/refunds`
CUS-17	View my loyalty account	`GET /api/v1/customer/loyalty/account`
CUS-18	View my points history	`GET /api/v1/customer/loyalty/history`
CUS-19	View offers available to me	`GET /api/v1/customer/loyalty/offers`
CUS-20	Send feedback	`POST /api/v1/customer/feedback`
CUS-21	View my feedback and replies	`GET /api/v1/customer/feedback`

CUS-1 View my profile

`GET /api/v1/customer/profile`

```
SELECT id, email, full_name, phone, active, created_at, last_login_at
FROM app_users
WHERE email = 'customer1@example.com';
```

CUS-2 Update my profile

`PUT /api/v1/customer/profile`

Only name and phone can change.

```
UPDATE app_users
SET full_name = 'Amal Perera', phone = '+94771111112', updated_at = SYSDATETIMEOFFSET()
WHERE email = 'customer1@example.com';
```

CUS-3 List my delivery addresses

`GET /api/v1/customer/addresses`

Soft-deleted addresses are hidden.

```
SELECT a.id, a.street_address, a.city, a.postal_code, a.phone_contact, a.is_default
FROM customer_addresses a
JOIN app_users u ON u.id = a.user_id
WHERE u.email = 'customer1@example.com' AND a.active = 1
ORDER BY a.is_default DESC, a.id;
```

CUS-4 Add a delivery address

`POST /api/v1/customer/addresses`

A new default address clears the previous default.

```
SET XACT_ABORT ON;
DECLARE @UserId BIGINT = (SELECT id FROM app_users WHERE email = 'customer1@example.com');
BEGIN TRANSACTION;
UPDATE customer_addresses SET is_default = 0, updated_at = SYSDATETIMEOFFSET()
WHERE user_id = @UserId AND is_default = 1;
INSERT INTO customer_addresses (user_id, street_address, city, postal_code, phone_contact, is_default, active)
VALUES (@UserId, '88 Flower Road', 'Colombo', '00700', '+94771111111', 1, 1);
COMMIT;
```

CUS-5 Edit a delivery address

`PUT /api/v1/customer/addresses/{id}`

```
UPDATE a
SET a.street_address = '90 Flower Road', a.postal_code = '00700', a.updated_at = SYSDATETIMEOFFSET()
FROM customer_addresses a
JOIN app_users u ON u.id = a.user_id
WHERE u.email = 'customer1@example.com' AND a.street_address = '88 Flower Road' AND a.active = 1;
```

CUS-6 Delete a delivery address (soft delete)

`DELETE /api/v1/customer/addresses/{id}`

Orders keep their own address snapshot, so the row is only deactivated.

```
UPDATE a
SET a.active = 0, a.is_default = 0, a.updated_at = SYSDATETIMEOFFSET()
FROM customer_addresses a
JOIN app_users u ON u.id = a.user_id
WHERE u.email = 'customer1@example.com' AND a.street_address = '90 Flower Road';

-- Make the original address the default again
UPDATE a SET a.is_default = 1, a.updated_at = SYSDATETIMEOFFSET()
FROM customer_addresses a JOIN app_users u ON u.id = a.user_id
WHERE u.email = 'customer1@example.com' AND a.street_address = '12 Galle Road' AND a.active = 1;
```

CUS-7 View my cart

`GET /api/v1/customer/cart`

Line totals use the current product price.

```
SELECT ci.id AS item_id, p.sku, p.name, p.unit, p.price, ci.quantity,
       p.price * ci.quantity AS line_total, i.quantity AS in_stock
FROM cart_items ci
JOIN carts c ON c.id = ci.cart_id
JOIN app_users u ON u.id = c.user_id
JOIN products p ON p.id = ci.product_id
LEFT JOIN inventory i ON i.product_id = p.id
WHERE u.email = 'customer2@example.com'
ORDER BY ci.id;

SELECT SUM(p.price * ci.quantity) AS subtotal, SUM(ci.quantity) AS item_count
FROM cart_items ci JOIN carts c ON c.id = ci.cart_id JOIN app_users u ON u.id = c.user_id
JOIN products p ON p.id = ci.product_id
WHERE u.email = 'customer2@example.com';
```

CUS-8 Add an item to my cart

`POST /api/v1/customer/cart/items`

Adds to the existing line if present; the total may not exceed stock; product must be active.

```
SET XACT_ABORT ON;
DECLARE @Email VARCHAR(255) = 'customer2@example.com', @Sku VARCHAR(100) = 'BK-BRD-001', @Qty INT = 2;
DECLARE @CartId BIGINT = (SELECT c.id FROM carts c JOIN app_users u ON u.id = c.user_id WHERE u.email = @Email);
DECLARE @ProductId BIGINT = (SELECT id FROM products WHERE sku = @Sku AND active = 1);
IF @ProductId IS NULL THROW 50003, 'Cannot add inactive product to cart.', 1;

DECLARE @Target INT = @Qty + ISNULL((SELECT quantity FROM cart_items WHERE cart_id = @CartId AND product_id =
    @ProductId), 0);
IF @Target > (SELECT quantity FROM inventory WHERE product_id = @ProductId)
    THROW 50004, 'Requested quantity exceeds available stock.', 1;

BEGIN TRANSACTION;
UPDATE cart_items SET quantity = @Target, updated_at = SYSDATETIMEOFFSET()
WHERE cart_id = @CartId AND product_id = @ProductId;
IF @@ROWCOUNT = 0
    INSERT INTO cart_items (cart_id, product_id, quantity) VALUES (@CartId, @ProductId, @Qty);
COMMIT;
```

CUS-9 Change a cart item quantity

`PUT /api/v1/customer/cart/items/{itemId}`

```
DECLARE @Email VARCHAR(255) = 'customer2@example.com', @Sku VARCHAR(100) = 'BV-WAT-003', @NewQty INT = 4;

UPDATE ci
SET ci.quantity = @NewQty, ci.updated_at = SYSDATETIMEOFFSET()
FROM cart_items ci
JOIN carts c ON c.id = ci.cart_id
JOIN app_users u ON u.id = c.user_id
JOIN products p ON p.id = ci.product_id
JOIN inventory i ON i.product_id = p.id
WHERE u.email = @Email AND p.sku = @Sku
  AND @NewQty > 0 AND @NewQty <= i.quantity;
```

CUS-10 Remove an item from my cart

`DELETE /api/v1/customer/cart/items/{itemId}`

```
DELETE ci
FROM cart_items ci
JOIN carts c ON c.id = ci.cart_id
JOIN app_users u ON u.id = c.user_id
JOIN products p ON p.id = ci.product_id
WHERE u.email = 'customer2@example.com' AND p.sku = 'BK-BRD-001';
```

CUS-11 Check out the cart (place an order)

`POST /api/v1/customer/orders/checkout`

One transaction: order + items (price snapshot), stock deduction with movement log, payment (CARD/PayPal simulations are PAID at once and CONFIRMED; COD is CONFIRMED/PENDING; EFT waits as AWAITING_PAYMENT), an unassigned PENDING delivery, and the cart is emptied. Offers/points are 0 here.

```
SET XACT_ABORT ON;
DECLARE @Email VARCHAR(255) = 'customer2@example.com', @Method VARCHAR(50) = 'COD',
        @DeliveryDate DATE = DATEADD(DAY, 2, CAST(SYSDATETIMEOFFSET() AS DATE)), @Slot VARCHAR(50) =
            'MORNING_09_12';
DECLARE @UserId BIGINT = (SELECT id FROM app_users WHERE email = @Email);
DECLARE @CartId BIGINT = (SELECT id FROM carts WHERE user_id = @UserId);
DECLARE @AddressId BIGINT = (SELECT TOP 1 id FROM customer_addresses WHERE user_id = @UserId AND active = 1 ORDER BY
    is_default DESC);
DECLARE @OrderNumber VARCHAR(100) = 'RM-' + UPPER(LEFT(REPLACE(CONVERT(VARCHAR(36), NEWID()), '-', ''), 8));

IF NOT EXISTS (SELECT 1 FROM cart_items WHERE cart_id = @CartId) THROW 50005, 'Cart is empty.', 1;

BEGIN TRANSACTION;
-- Lock stock rows and verify availability
IF EXISTS (SELECT 1 FROM cart_items ci JOIN inventory i WITH (UPDLOCK, ROWLOCK) ON i.product_id = ci.product_id
           WHERE ci.cart_id = @CartId AND ci.quantity > i.quantity)
    THROW 50006, 'Insufficient stock for one or more cart items.', 1;

DECLARE @Subtotal DECIMAL(12,2) = (SELECT SUM(p.price * ci.quantity) FROM cart_items ci
                                   JOIN products p ON p.id = ci.product_id WHERE ci.cart_id = @CartId);
DECLARE @Paid BIT = CASE WHEN @Method IN ('CARD_SIMULATION', 'PAYPAL_SIMULATION') THEN 1 ELSE 0 END;

INSERT INTO orders (order_number, user_id, status, delivery_date, delivery_slot, delivery_street, delivery_city,
                    delivery_postal_code, delivery_phone, subtotal, total_amount)
SELECT @OrderNumber, @UserId, CASE WHEN @Method = 'EFT' THEN 'AWAITING_PAYMENT' ELSE 'CONFIRMED' END,
       @DeliveryDate, @Slot, a.street_address, a.city, a.postal_code, a.phone_contact, @Subtotal, @Subtotal
FROM customer_addresses a WHERE a.id = @AddressId;
DECLARE @OrderId BIGINT = SCOPE_IDENTITY();
INSERT INTO order_items (order_id, product_id, product_name_snapshot, unit_price_snapshot, quantity, line_total)
SELECT @OrderId, p.id, p.name, p.price, ci.quantity, p.price * ci.quantity
FROM cart_items ci JOIN products p ON p.id = ci.product_id WHERE ci.cart_id = @CartId;

UPDATE i SET i.quantity = i.quantity - ci.quantity, i.version = i.version + 1, i.updated_at = SYSDATETIMEOFFSET()
FROM inventory i JOIN cart_items ci ON ci.product_id = i.product_id WHERE ci.cart_id = @CartId;

INSERT INTO stock_movement_history (product_id, quantity_delta, movement_type, reference_id, actor_id, actor_email)
SELECT ci.product_id, -ci.quantity, 'CHECKOUT_DEDUCTION', @OrderNumber, @UserId, @Email
FROM cart_items ci WHERE ci.cart_id = @CartId;

INSERT INTO payments (order_id, payment_method, payment_status, amount, transaction_reference, notes)
VALUES (@OrderId, @Method, CASE WHEN @Paid = 1 THEN 'PAID' ELSE 'PENDING' END, @Subtotal,
```

```
        @Method + '-' + LOWER(LEFT(REPLACE(CONVERT(VARCHAR(36), NEWID()), '-', ''), 8)),
        CASE WHEN @Paid = 1 THEN 'LOCAL SIMULATION - no real card/PayPal transaction occurred'
             ELSE 'Initial checkout payment record' END);

INSERT INTO deliveries (order_id, delivery_status, scheduled_date, scheduled_slot)
VALUES (@OrderId, 'PENDING', @DeliveryDate, @Slot);

DELETE FROM cart_items WHERE cart_id = @CartId;
COMMIT;

SELECT @OrderNumber AS order_number, @Subtotal AS total_amount;
```

CUS-12 List my orders

`GET /api/v1/customer/orders`

Newest first, with payment and active bill.

```
SELECT o.id, o.order_number, o.status, o.delivery_date, o.delivery_slot, o.total_amount,
       p.payment_method, p.payment_status, b.bill_number, o.created_at
FROM orders o
JOIN app_users u ON u.id = o.user_id
LEFT JOIN payments p ON p.order_id = o.id
LEFT JOIN bills b ON b.order_id = o.id AND b.status = 'ACTIVE'
WHERE u.email = 'customer1@example.com'
ORDER BY o.created_at DESC;
```

CUS-13 View one order with its items

`GET /api/v1/customer/orders/{id}`

```
DECLARE @OrderNumber VARCHAR(100) = 'RM-DEMO-0001';

SELECT o.order_number, o.status, o.delivery_date, o.delivery_slot,
       o.delivery_street, o.delivery_city, o.delivery_postal_code,
       o.subtotal, o.offer_discount, o.point_discount, o.tax_percentage, o.tax_amount,
       o.delivery_charge, o.total_amount, p.payment_method, p.payment_status, d.delivery_status
FROM orders o
LEFT JOIN payments p ON p.order_id = o.id
LEFT JOIN deliveries d ON d.order_id = o.id
WHERE o.order_number = @OrderNumber;

SELECT i.product_name_snapshot, i.unit_price_snapshot, i.quantity, i.line_total
FROM order_items i JOIN orders o ON o.id = i.order_id
WHERE o.order_number = @OrderNumber;
```

CUS-14 Cancel my order

`POST /api/v1/customer/orders/{id}/cancel`

Only AWAITING_PAYMENT/CONFIRMED orders whose delivery is still PENDING and that are not yet PAID. Restores stock, cancels the delivery, fails the pending payment and voids any bill.

```
SET XACT_ABORT ON;
DECLARE @OrderId BIGINT = (SELECT TOP 1 o.id FROM orders o JOIN app_users u ON u.id = o.user_id
                           WHERE u.email = 'customer2@example.com' AND o.order_number NOT LIKE 'RM-DEMO-%'
                           ORDER BY o.id DESC);   -- the order placed in CUS-11
DECLARE @ActorId BIGINT = (SELECT user_id FROM orders WHERE id = @OrderId);
DECLARE @ActorEmail VARCHAR(255) = (SELECT email FROM app_users WHERE id = @ActorId);

IF NOT EXISTS (SELECT 1 FROM orders o JOIN deliveries d ON d.order_id = o.id JOIN payments p ON p.order_id = o.id
               WHERE o.id = @OrderId AND o.status IN ('AWAITING_PAYMENT', 'CONFIRMED')
                 AND d.delivery_status = 'PENDING' AND p.payment_status <> 'PAID')
    THROW 50007, 'Order cannot be cancelled online (assigned, paid or already closed).', 1;

BEGIN TRANSACTION;
UPDATE orders SET status = 'CANCELLED', updated_at = SYSDATETIMEOFFSET() WHERE id = @OrderId;
UPDATE i SET i.quantity = i.quantity + oi.quantity, i.version = i.version + 1, i.updated_at = SYSDATETIMEOFFSET()
FROM inventory i JOIN order_items oi ON oi.product_id = i.product_id WHERE oi.order_id = @OrderId;

INSERT INTO stock_movement_history (product_id, quantity_delta, movement_type, reference_id, actor_id, actor_email)
SELECT oi.product_id, oi.quantity, 'CANCELLATION_RESTORATION', 'CANCEL_' + o.order_number, @ActorId, @ActorEmail
FROM order_items oi JOIN orders o ON o.id = oi.order_id WHERE oi.order_id = @OrderId;

UPDATE deliveries SET delivery_status = 'CANCELLED', updated_at = SYSDATETIMEOFFSET() WHERE order_id = @OrderId;

UPDATE bills SET status = 'VOIDED', voided_at = SYSDATETIMEOFFSET(), voided_by_id = @ActorId,
                 void_reason = 'Order cancelled before payment', updated_at = SYSDATETIMEOFFSET()
WHERE order_id = @OrderId AND status = 'ACTIVE';
UPDATE payments SET payment_status = 'FAILED', notes = 'Order cancelled before payment: ', updated_at =
    SYSDATETIMEOFFSET()
WHERE order_id = @OrderId AND payment_status = 'PENDING';
COMMIT;
```

CUS-15 Request a refund

`POST /api/v1/customer/refunds`

One refund request per order; the order must belong to the customer and be paid.

```
DECLARE @Email VARCHAR(255) = 'customer1@example.com', @OrderNumber VARCHAR(100) = 'RM-DEMO-0003';

INSERT INTO refund_requests (order_id, user_id, refund_status, refund_amount, reason)
SELECT o.id, o.user_id, 'PENDING', o.total_amount, 'Cheddar cheese was past its best-before date.'
FROM orders o
JOIN app_users u ON u.id = o.user_id
JOIN payments p ON p.order_id = o.id
WHERE o.order_number = @OrderNumber AND u.email = @Email AND p.payment_status = 'PAID'
  AND NOT EXISTS (SELECT 1 FROM refund_requests r WHERE r.order_id = o.id);
```

CUS-16 List my refund requests

`GET /api/v1/customer/refunds`

```
SELECT r.id, o.order_number, r.refund_status, r.refund_amount, r.refunded_amount, r.reason,
       r.cashier_notes, r.created_at, r.processed_at
FROM refund_requests r
JOIN orders o ON o.id = r.order_id
JOIN app_users u ON u.id = r.user_id
WHERE u.email = 'customer1@example.com'
ORDER BY r.created_at DESC;
```

CUS-17 View my loyalty account

`GET /api/v1/customer/loyalty/account`

No tier shows as BRONZE.

```
SELECT la.current_points, la.lifetime_points, ISNULL(t.tier_name, 'BRONZE') AS tier_name
FROM loyalty_accounts la
JOIN app_users u ON u.id = la.user_id
LEFT JOIN loyalty_tiers t ON t.id = la.current_tier_id
WHERE u.email = 'customer2@example.com';
```

CUS-18 View my points history

`GET /api/v1/customer/loyalty/history`

```
SELECT t.created_at, t.points_delta, t.balance_after, t.reason, t.reference_id
FROM reward_transactions t
JOIN loyalty_accounts la ON la.id = t.loyalty_account_id
JOIN app_users u ON u.id = la.user_id
WHERE u.email = 'customer2@example.com'
ORDER BY t.created_at DESC, t.id DESC;
```

CUS-19 View offers available to me

`GET /api/v1/customer/loyalty/offers`

Active offers valid today; tier-restricted offers need lifetime points >= that tier's minimum.

```
DECLARE @Lifetime INT = ISNULL((SELECT la.lifetime_points FROM loyalty_accounts la
                                JOIN app_users u ON u.id = la.user_id
                                WHERE u.email = 'customer2@example.com'), 0);
DECLARE @Today DATE = CAST(SYSDATETIMEOFFSET() AS DATE);

SELECT o.code, o.title, o.discount_percentage, o.min_order_amount, o.start_date, o.end_date,
       t.tier_name AS eligible_tier
FROM offers o
LEFT JOIN loyalty_tiers t ON t.id = o.eligible_tier_id
WHERE o.active = 1 AND @Today BETWEEN o.start_date AND o.end_date
  AND (o.eligible_tier_id IS NULL OR @Lifetime >= t.minimum_points)
ORDER BY o.discount_percentage;
```

CUS-20 Send feedback

`POST /api/v1/customer/feedback`

```
INSERT INTO feedbacks (user_id, category, subject, message, status)
SELECT id, 'APP', 'Checkout was quick', 'Placing my order took less than a minute.', 'PENDING'
FROM app_users WHERE email = 'customer1@example.com';
```

CUS-21 View my feedback and replies

`GET /api/v1/customer/feedback`

```
SELECT f.id, f.category, f.subject, f.status, f.created_at,
       r.reply_message, m.full_name AS replied_by, r.created_at AS replied_at
FROM feedbacks f
JOIN app_users u ON u.id = f.user_id
LEFT JOIN feedback_replies r ON r.feedback_id = f.id
LEFT JOIN app_users m ON m.id = r.replied_by_id
WHERE u.email = 'customer2@example.com'
ORDER BY f.created_at DESC, r.created_at;
```

Cashier (payments, refunds, COD, billing) (16 operations)

Demo login: cashier@rapidmart.local / DemoStaff21! (staff portal; MANAGER also has access)

Cashiers verify EFT transfers, confirm cash handed over by drivers, process refunds and run billing (tax % and delivery charge are entered per order, then the bill is generated). Demo: RM-DEMO-0002 is an EFT awaiting verification, RM-DEMO-0001 has a pending refund, RM-DEMO-0004/0005 await bill generation.

ID	Operation	Endpoint
CSH-1	EFT payments awaiting verification	`GET /api/v1/cashier/payments/pending`
CSH-2	Approve (or reject) an EFT payment	`POST /api/v1/cashier/payments/{orderId}/verify-eft`
CSH-3	Unconfirmed COD cash hand-overs	`GET /api/v1/cashier/cod-settlements`
CSH-4	Confirm COD cash received	`POST /api/v1/cashier/cod-settlements/{id}/confirm`
CSH-5	List refund requests	`GET /api/v1/cashier/refunds`
CSH-6	Approve a refund	`POST /api/v1/cashier/refunds/{id}/process`
CSH-7	Refund status history	`GET /api/v1/cashier/refunds/{id}/history`
CSH-8	Orders awaiting bill generation	`GET /api/v1/billing/pending`
CSH-9	Preview, then finalize charges and generate the bill	`POST /api/v1/billing/orders/{orderId}/preview · /finalize`
CSH-10	Search bills	`GET /api/v1/billing/search?query=&status=`
CSH-11	Bill detail (header, lines, live payment/refund status)	`GET /api/v1/billing/{billNumber}`
CSH-12	Current active bill of an order	`GET /api/v1/billing/order/{orderId}`
CSH-13	Edit bill notes	`PATCH /api/v1/billing/{billNumber}`
CSH-14	Void a bill	`POST /api/v1/billing/{billNumber}/void`
CSH-15	Reissue a bill	`POST /api/v1/billing/reissue/{orderId}`
CSH-16	Log a bill e-mail	`POST /api/v1/billing/{billNumber}/email`

CSH-1 EFT payments awaiting verification

`GET /api/v1/cashier/payments/pending`

```
SELECT o.id AS order_id, o.order_number, u.full_name, u.email, p.amount, p.payment_status,
       p.transaction_reference, p.notes, p.created_at
FROM payments p
JOIN orders o ON o.id = p.order_id
JOIN app_users u ON u.id = o.user_id
WHERE p.payment_method = 'EFT' AND p.payment_status IN ('PENDING', 'FAILED')
ORDER BY p.created_at;
```

CSH-2 Approve (or reject) an EFT payment

`POST /api/v1/cashier/payments/{orderId}/verify-eft`

Approve: payment PAID + verified stamp, order CONFIRMED. Reject: payment FAILED. A note is required. If the order is already finalized, the app also issues its bill (see CSH-9).

```
SET XACT_ABORT ON;
DECLARE @OrderNumber VARCHAR(100) = 'RM-DEMO-0002', @Approve BIT = 1,
        @Notes VARCHAR(500) = 'Transfer matched on bank statement';
DECLARE @CashierId BIGINT = (SELECT id FROM app_users WHERE email = 'cashier@rapidmart.local');
DECLARE @OrderId BIGINT = (SELECT id FROM orders WHERE order_number = @OrderNumber);

IF NOT EXISTS (SELECT 1 FROM payments p JOIN orders o ON o.id = p.order_id
               WHERE o.id = @OrderId AND p.payment_method = 'EFT' AND o.status = 'AWAITING_PAYMENT'
                 AND p.payment_status IN ('PENDING', 'FAILED'))
    THROW 50008, 'Not an EFT payment awaiting verification.', 1;

BEGIN TRANSACTION;
IF @Approve = 1
BEGIN
    UPDATE payments SET payment_status = 'PAID', notes = @Notes, verified_by_id = @CashierId,
                        verified_at = SYSDATETIMEOFFSET(), updated_at = SYSDATETIMEOFFSET()
    WHERE order_id = @OrderId;
    UPDATE orders SET status = 'CONFIRMED', updated_at = SYSDATETIMEOFFSET() WHERE id = @OrderId;
END
ELSE
    UPDATE payments SET payment_status = 'FAILED', notes = @Notes, updated_at = SYSDATETIMEOFFSET()
    WHERE order_id = @OrderId;
COMMIT;
```

CSH-3 Unconfirmed COD cash hand-overs

`GET /api/v1/cashier/cod-settlements`

Created when a driver completes a COD delivery (see DST-4).

```
SELECT s.id, o.order_number, d.full_name AS driver, s.collected_amount, s.created_at
FROM cod_settlements s
JOIN deliveries dl ON dl.id = s.delivery_id
JOIN orders o ON o.id = dl.order_id
JOIN app_users d ON d.id = s.delivery_staff_id
WHERE s.confirmed_by_cashier = 0
ORDER BY s.created_at;
```

CSH-4 Confirm COD cash received

`POST /api/v1/cashier/cod-settlements/{id}/confirm`

Also the COD payment verification: PENDING payment becomes PAID. Idempotent.

```
SET XACT_ABORT ON;
DECLARE @OrderNumber VARCHAR(100) = 'RM-DEMO-0005';
DECLARE @CashierId BIGINT = (SELECT id FROM app_users WHERE email = 'cashier@rapidmart.local');
DECLARE @SettlementId BIGINT = (SELECT s.id FROM cod_settlements s JOIN deliveries d ON d.id = s.delivery_id
                                JOIN orders o ON o.id = d.order_id WHERE o.order_number = @OrderNumber);

BEGIN TRANSACTION;
UPDATE cod_settlements SET confirmed_by_cashier = 1, cashier_id = @CashierId, settled_at = SYSDATETIMEOFFSET()
WHERE id = @SettlementId AND confirmed_by_cashier = 0;

UPDATE p SET p.payment_status = 'PAID', p.verified_by_id = @CashierId, p.verified_at = SYSDATETIMEOFFSET(),
             p.updated_at = SYSDATETIMEOFFSET()
FROM payments p JOIN orders o ON o.id = p.order_id
WHERE o.order_number = @OrderNumber AND p.payment_status = 'PENDING';
COMMIT;
```

CSH-5 List refund requests

`GET /api/v1/cashier/refunds`

```
SELECT r.id, o.order_number, u.full_name, u.email, r.refund_status, r.refund_amount,
       r.refunded_amount, r.reason, r.created_at, pr.email AS processed_by
FROM refund_requests r
JOIN orders o ON o.id = r.order_id
JOIN app_users u ON u.id = r.user_id
LEFT JOIN app_users pr ON pr.id = r.processed_by_id
ORDER BY CASE r.refund_status WHEN 'PENDING' THEN 0 ELSE 1 END, r.created_at DESC;
```

CSH-6 Approve a refund

`POST /api/v1/cashier/refunds/{id}/process`

Payment must be PAID and the order not in transit; the amount may not exceed the active bill. Delivered orders have their earned points reversed (never below zero); undelivered orders are cancelled and restocked (as in CUS-14). Writes status history and an audit entry. Rejecting only sets REJECTED + history.

```
SET XACT_ABORT ON;
DECLARE @OrderNumber VARCHAR(100) = 'RM-DEMO-0001', @Amount DECIMAL(12,2) = 220.00,
        @Reason VARCHAR(MAX) = 'Damaged item confirmed; refunded in cash';
DECLARE @CashierId BIGINT = (SELECT id FROM app_users WHERE email = 'cashier@rapidmart.local');
DECLARE @OrderId BIGINT, @RefundId BIGINT, @UserId BIGINT, @Status VARCHAR(50);
SELECT @OrderId = o.id, @RefundId = r.id, @UserId = r.user_id, @Status = o.status
FROM refund_requests r JOIN orders o ON o.id = r.order_id
WHERE o.order_number = @OrderNumber AND r.refund_status = 'PENDING';

IF @RefundId IS NULL THROW 50009, 'No pending refund for this order.', 1;
IF NOT EXISTS (SELECT 1 FROM payments WHERE order_id = @OrderId AND payment_status = 'PAID')
    THROW 50010, 'Only paid payments can be refunded.', 1;
IF @Status = 'OUT_FOR_DELIVERY' THROW 50011, 'Wait until the delivery is completed or has failed.', 1;
IF @Amount > ISNULL((SELECT amount FROM bills WHERE order_id = @OrderId AND status = 'ACTIVE'),
                    (SELECT total_amount FROM orders WHERE id = @OrderId))
    THROW 50012, 'Refunded amount cannot exceed the bill amount.', 1;

BEGIN TRANSACTION;
DECLARE @Reversed INT = 0;
IF @Status = 'DELIVERED'
BEGIN
    DECLARE @AccountId BIGINT, @Balance INT;
    SELECT @AccountId = id, @Balance = current_points FROM loyalty_accounts WHERE user_id = @UserId;
    SET @Reversed = ISNULL((SELECT SUM(points_delta) FROM reward_transactions
                            WHERE reference_id = 'ORDER_' + @OrderNumber AND reason = 'EARNED_ORDER_COMPLETED'), 0);
    IF @Reversed > @Balance SET @Reversed = @Balance;
    IF @Reversed > 0
    BEGIN
```

```
        UPDATE loyalty_accounts SET current_points = current_points - @Reversed, version = version + 1,
                                    updated_at = SYSDATETIMEOFFSET() WHERE id = @AccountId;
        INSERT INTO reward_transactions (loyalty_account_id, points_delta, balance_after, reason, reference_id,
            actor_id)
        VALUES (@AccountId, -@Reversed, @Balance - @Reversed, 'REFUND_POINTS_REVERSAL', 'ORDER_' + @OrderNumber,
            @UserId);
    END
END
-- (undelivered orders: run the cancellation block from CUS-14 here)

UPDATE payments SET payment_status = 'REFUNDED', updated_at = SYSDATETIMEOFFSET() WHERE order_id = @OrderId;

UPDATE refund_requests
SET refund_status = 'REFUNDED', refunded_amount = @Amount, points_reversed = @Reversed, cashier_notes = @Reason,
    processed_by_id = @CashierId, processed_at = SYSDATETIMEOFFSET(), updated_at = SYSDATETIMEOFFSET()
WHERE id = @RefundId;

INSERT INTO refund_status_history (refund_request_id, previous_status, new_status, refund_amount, reason,
    changed_by_id)
VALUES (@RefundId, 'PENDING', 'REFUNDED', @Amount, @Reason, @CashierId);

INSERT INTO audit_logs (action, actor_email, target_reference, details)
VALUES ('REFUND_APPROVED', 'cashier@rapidmart.local', 'ORDER_' + @OrderNumber,
        'Refund of LKR ' + CAST(@Amount AS VARCHAR(20)) + ' recorded as completed externally. Points reversed: '
        + CAST(@Reversed AS VARCHAR(10)));
COMMIT;
```

CSH-7 Refund status history

`GET /api/v1/cashier/refunds/{id}/history`

Append-only, newest first.

```
SELECT h.changed_at, h.previous_status, h.new_status, h.refund_amount, h.reason, u.email AS changed_by
FROM refund_status_history h
JOIN refund_requests r ON r.id = h.refund_request_id
JOIN orders o ON o.id = r.order_id
JOIN app_users u ON u.id = h.changed_by_id
WHERE o.order_number = 'RM-DEMO-0001'
ORDER BY h.changed_at DESC, h.id DESC;
```

CSH-8 Orders awaiting bill generation

`GET /api/v1/billing/pending`

Non-cancelled orders that have never had a bill, any payment method.

```
SELECT o.id, o.order_number, u.full_name, u.email, o.status, p.payment_method, p.payment_status,
       o.subtotal, o.offer_discount, o.point_discount, o.total_amount, o.is_finalized, p.created_at
FROM payments p
JOIN orders o ON o.id = p.order_id
JOIN app_users u ON u.id = o.user_id
WHERE o.status <> 'CANCELLED'
  AND NOT EXISTS (SELECT 1 FROM bills b WHERE b.order_id = o.id)
ORDER BY p.created_at DESC;
```

CSH-9 Preview, then finalize charges and generate the bill

`POST /api/v1/billing/orders/{orderId}/preview · /finalize`

Net = (subtotal - discounts) + tax% of that + delivery charge. Unpaid orders: the payment amount becomes the net. Paid orders: net must equal the verified payment amount. One ACTIVE bill per order (unique index).

```
SET XACT_ABORT ON;
DECLARE @OrderNumber VARCHAR(100) = 'RM-DEMO-0004', @TaxPct DECIMAL(5,2) = 5.00, @Delivery DECIMAL(12,2) = 150.00;
DECLARE @CashierId BIGINT = (SELECT id FROM app_users WHERE email = 'cashier@rapidmart.local');

-- Preview (no changes)
SELECT o.order_number, d.discounted, @TaxPct AS tax_pct, CAST(ROUND(d.discounted * @TaxPct / 100, 2) AS
    DECIMAL(12,2)) AS tax_amount,
       @Delivery AS delivery_charge,
       CAST(d.discounted + ROUND(d.discounted * @TaxPct / 100, 2) + @Delivery AS DECIMAL(12,2)) AS net_amount,
       p.payment_status, p.amount AS payment_amount
FROM orders o JOIN payments p ON p.order_id = o.id
CROSS APPLY (SELECT CASE WHEN o.subtotal - o.offer_discount - o.point_discount < 0 THEN 0
                         ELSE o.subtotal - o.offer_discount - o.point_discount END AS discounted) d
WHERE o.order_number = @OrderNumber;
```

```
-- Finalize + generate
DECLARE @OrderId BIGINT, @Discounted DECIMAL(12,2), @PayStatus VARCHAR(50), @PayAmount DECIMAL(12,2);
SELECT @OrderId = o.id, @PayStatus = p.payment_status, @PayAmount = p.amount,
       @Discounted = CASE WHEN o.subtotal - o.offer_discount - o.point_discount < 0 THEN 0
                          ELSE o.subtotal - o.offer_discount - o.point_discount END
FROM orders o JOIN payments p ON p.order_id = o.id
WHERE o.order_number = @OrderNumber AND o.status <> 'CANCELLED';
DECLARE @Tax DECIMAL(12,2) = ROUND(@Discounted * @TaxPct / 100, 2);
DECLARE @Net DECIMAL(12,2) = @Discounted + @Tax + @Delivery;

IF @OrderId IS NULL THROW 50013, 'Order not found or cancelled.', 1;
IF EXISTS (SELECT 1 FROM bills WHERE order_id = @OrderId) THROW 50014, 'Bill already generated; use reissue.', 1;
IF @PayStatus = 'PAID' AND @PayAmount <> @Net THROW 50015, 'Net amount does not match the verified payment.', 1;
BEGIN TRANSACTION;
IF @PayStatus <> 'PAID'
    UPDATE payments SET amount = @Net, updated_at = SYSDATETIMEOFFSET() WHERE order_id = @OrderId;

UPDATE orders SET tax_percentage = @TaxPct, tax_amount = @Tax, delivery_charge = @Delivery, total_amount = @Net,
                  is_finalized = 1, finalized_at = SYSDATETIMEOFFSET(), finalized_by_id = @CashierId,
                  updated_at = SYSDATETIMEOFFSET()
WHERE id = @OrderId;

INSERT INTO bills (bill_number, order_id, amount, subtotal, offer_discount, point_discount,
                   tax_percentage, tax_amount, delivery_charge, issued_by_id)
SELECT 'BILL-' + UPPER(LEFT(REPLACE(CONVERT(VARCHAR(36), NEWID()), '-', ''), 8)), o.id, @Net,
       o.subtotal, o.offer_discount, o.point_discount, @TaxPct, @Tax, @Delivery, @CashierId
FROM orders o WHERE o.id = @OrderId;
COMMIT;
```

CSH-10 Search bills

`GET /api/v1/billing/search?query=&status=`

Matches bill number, order number or customer email.

```
DECLARE @Query VARCHAR(100) = 'customer1', @Status VARCHAR(20) = 'ACTIVE';

SELECT b.id, b.bill_number, o.order_number, u.email AS customer, b.amount, b.status, b.issued_at
FROM bills b
JOIN orders o ON o.id = b.order_id
JOIN app_users u ON u.id = o.user_id
WHERE (@Status IS NULL OR b.status = @Status)
  AND (@Query IS NULL OR @Query = ''
       OR LOWER(b.bill_number) LIKE '%' + LOWER(@Query) + '%'
       OR LOWER(o.order_number) LIKE '%' + LOWER(@Query) + '%'
       OR LOWER(u.email) LIKE '%' + LOWER(@Query) + '%')
ORDER BY b.issued_at DESC
OFFSET 0 ROWS FETCH NEXT 20 ROWS ONLY;
```

CSH-11 Bill detail (header, lines, live payment/refund status)

`GET /api/v1/billing/{billNumber}`

Charge snapshots fall back to the order values for older bills.

```
DECLARE @BillNumber VARCHAR(100) = 'BILL-DEMO-0001';

SELECT b.bill_number, b.status, o.order_number, u.full_name, u.email, u.phone,
       o.delivery_street, o.delivery_city, o.delivery_postal_code,
       ISNULL(b.subtotal, o.subtotal) AS subtotal, ISNULL(b.offer_discount, o.offer_discount) AS offer_discount,
       ISNULL(b.point_discount, o.point_discount) AS point_discount, ISNULL(b.tax_percentage, o.tax_percentage) AS
           tax_pct,
       ISNULL(b.tax_amount, o.tax_amount) AS tax_amount, ISNULL(b.delivery_charge, o.delivery_charge) AS
           delivery_charge,
       b.amount AS total, p.payment_method, p.payment_status, r.refund_status, r.refunded_amount,
       b.issued_at, ib.email AS issued_by, b.notes, b.voided_at, vb.email AS voided_by, b.void_reason
FROM bills b
JOIN orders o ON o.id = b.order_id
JOIN app_users u ON u.id = o.user_id
LEFT JOIN payments p ON p.order_id = o.id
LEFT JOIN refund_requests r ON r.order_id = o.id
LEFT JOIN app_users ib ON ib.id = b.issued_by_id
LEFT JOIN app_users vb ON vb.id = b.voided_by_id
WHERE b.bill_number = @BillNumber;

SELECT i.product_name_snapshot, i.unit_price_snapshot, i.quantity, i.line_total
FROM order_items i JOIN bills b ON b.order_id = i.order_id
WHERE b.bill_number = @BillNumber;
```

CSH-12 Current active bill of an order

`GET /api/v1/billing/order/{orderId}`

```
SELECT b.id, b.bill_number, o.order_number, u.email, b.amount, b.status, b.issued_at
FROM bills b JOIN orders o ON o.id = b.order_id JOIN app_users u ON u.id = o.user_id
WHERE o.order_number = 'RM-DEMO-0003' AND b.status = 'ACTIVE';
```

CSH-13 Edit bill notes

`PATCH /api/v1/billing/{billNumber}`

Notes are the only editable field of an issued bill.

```
UPDATE bills SET notes = 'Customer asked for a printed copy', updated_at = SYSDATETIMEOFFSET()
WHERE bill_number = 'BILL-DEMO-0003';
```

CSH-14 Void a bill

`POST /api/v1/billing/{billNumber}/void`

ACTIVE -> VOIDED; idempotent.

```
UPDATE bills
SET status = 'VOIDED', voided_at = SYSDATETIMEOFFSET(),
    voided_by_id = (SELECT id FROM app_users WHERE email = 'cashier@rapidmart.local'),
    void_reason = 'Printed with wrong delivery charge', updated_at = SYSDATETIMEOFFSET()
WHERE bill_number = (SELECT b.bill_number FROM bills b JOIN orders o ON o.id = b.order_id
                     WHERE o.order_number = 'RM-DEMO-0004' AND b.status = 'ACTIVE')
  AND status = 'ACTIVE';
```

CSH-15 Reissue a bill

`POST /api/v1/billing/reissue/{orderId}`

Voids the active bill (if any) and issues a new one. The order must be PAID, finalized, and the payment amount must equal the order total. Demo: RM-DEMO-0005 qualifies after DST-4, CSH-4 and CSH-9 with 0% tax / 0 delivery (net 1080 = cash collected); until then the guard stops it.

```
SET XACT_ABORT ON;
DECLARE @OrderNumber VARCHAR(100) = 'RM-DEMO-0005';
DECLARE @CashierId BIGINT = (SELECT id FROM app_users WHERE email = 'cashier@rapidmart.local');
DECLARE @OrderId BIGINT = (SELECT id FROM orders WHERE order_number = @OrderNumber);

IF NOT EXISTS (SELECT 1 FROM orders o JOIN payments p ON p.order_id = o.id
               WHERE o.id = @OrderId AND p.payment_status = 'PAID' AND o.is_finalized = 1 AND p.amount =
                   o.total_amount)
    THROW 50016, 'Reissue needs a paid, finalized order whose payment matches the total.', 1;

BEGIN TRANSACTION;
UPDATE bills SET status = 'VOIDED', voided_at = SYSDATETIMEOFFSET(), voided_by_id = @CashierId,
                 void_reason = 'Reissued', updated_at = SYSDATETIMEOFFSET()
WHERE order_id = @OrderId AND status = 'ACTIVE';
INSERT INTO bills (bill_number, order_id, amount, subtotal, offer_discount, point_discount,
                   tax_percentage, tax_amount, delivery_charge, issued_by_id)
SELECT 'BILL-' + UPPER(LEFT(REPLACE(CONVERT(VARCHAR(36), NEWID()), '-', ''), 8)), o.id, o.total_amount,
       o.subtotal, o.offer_discount, o.point_discount, o.tax_percentage, o.tax_amount, o.delivery_charge, @CashierId
FROM orders o WHERE o.id = @OrderId;
COMMIT;
```

CSH-16 Log a bill e-mail

`POST /api/v1/billing/{billNumber}/email`

The PDF and SMTP send happen in the app; the database only records the notification outcome.

```
INSERT INTO notification_logs (recipient, channel, subject, message, status)
SELECT u.email, 'EMAIL', 'RapidMart Bill ' + b.bill_number,
       'Your bill ' + b.bill_number + ' for order ' + o.order_number + ' is attached.', 'NOT_SENT'
FROM bills b JOIN orders o ON o.id = b.order_id JOIN app_users u ON u.id = o.user_id
WHERE b.bill_number = 'BILL-DEMO-0003';
```

Inventory staff (14 operations)

Demo login: inventory@rapidmart.local / DemoStaff21! (staff portal; MANAGER also has access)

Inventory staff maintain categories and products, set stock limits, raise and receive restock requests, and audit stock movements. Demo: Instant Coffee (BV-COF-004) is out of stock with a PENDING restock request for 50.

ID	Operation	Endpoint
INV-1	List categories	`GET /api/v1/inventory/categories`
INV-2	Create a category	`POST /api/v1/inventory/categories`
INV-3	Edit / deactivate a category	`PUT /api/v1/inventory/categories/{id}`
INV-4	List products with stock	`GET /api/v1/inventory/products`
INV-5	Create a product with opening stock	`POST /api/v1/inventory/products`
INV-6	Edit a product	`PUT /api/v1/inventory/products/{id}`
INV-7	Deactivate a product (soft delete)	`PUT /api/v1/inventory/products/{id}/deactivate · DELETE`
INV-8	Low-stock report	`GET /api/v1/inventory/low-stock`
INV-9	Set stock limits	`PUT /api/v1/inventory/products/{productId}/limits`
INV-10	Raise a restock request	`POST /api/v1/inventory/restock-requests`
INV-11	List restock requests	`GET /api/v1/inventory/restock-requests`
INV-12	Approve or reject a restock request	`PUT /api/v1/inventory/restock-requests/{id}/approve · /reject`
INV-13	Receive stock for an approved request	`POST /api/v1/inventory/restock-requests/{id}/receive`
INV-14	Stock movement history	`GET /api/v1/inventory/movements?productId=`

INV-1 List categories

`GET /api/v1/inventory/categories`

Includes inactive categories.

```
SELECT c.id, c.name, c.description, c.active,
       (SELECT COUNT(*) FROM products p WHERE p.category_id = c.id) AS products
FROM categories c
ORDER BY c.name;
```

INV-2 Create a category

`POST /api/v1/inventory/categories`

Names are unique.

```
INSERT INTO categories (name, description, active)
SELECT 'Frozen Foods', 'Ice cream, frozen vegetables and ready meals', 1
WHERE NOT EXISTS (SELECT 1 FROM categories WHERE name = 'Frozen Foods');
```

INV-3 Edit / deactivate a category

`PUT /api/v1/inventory/categories/{id}`

```
UPDATE categories
SET description = 'Frozen desserts, vegetables and ready meals', active = 1, updated_at = SYSDATETIMEOFFSET()
WHERE name = 'Frozen Foods';
```

INV-4 List products with stock

`GET /api/v1/inventory/products`

Includes inactive products.

```
SELECT p.id, p.sku, p.name, c.name AS category, p.price, p.unit, p.active,
       i.quantity, i.minimum_level, i.maximum_level
FROM products p
JOIN categories c ON c.id = p.category_id
LEFT JOIN inventory i ON i.product_id = p.id
ORDER BY p.id
OFFSET 0 ROWS FETCH NEXT 50 ROWS ONLY;
```

INV-5 Create a product with opening stock

`POST /api/v1/inventory/products`

SKU unique, price > 0, opening stock 0-1000 logged as INITIAL_STOCK.

```
SET XACT_ABORT ON;
DECLARE @InitialStock INT = 40;
BEGIN TRANSACTION;
INSERT INTO products (sku, name, description, price, unit, category_id, active)
SELECT 'FZ-ICE-001', 'Vanilla Ice Cream', 'Creamy vanilla ice cream tub', 850.00, '1 L', id, 1
FROM categories WHERE name = 'Frozen Foods';
DECLARE @ProductId BIGINT = SCOPE_IDENTITY();

INSERT INTO inventory (product_id, quantity, minimum_level, maximum_level) VALUES (@ProductId, @InitialStock, 10,
    1000);
IF @InitialStock > 0
    INSERT INTO stock_movement_history (product_id, quantity_delta, movement_type, reference_id, actor_id,
        actor_email)
    SELECT @ProductId, @InitialStock, 'INITIAL_STOCK', 'PRODUCT_CREATED', id, email
    FROM app_users WHERE email = 'inventory@rapidmart.local';
COMMIT;
```

INV-6 Edit a product

`PUT /api/v1/inventory/products/{id}`

Stock is not changed here. The version column drives optimistic locking.

```
UPDATE products
SET name = 'Vanilla Ice Cream Tub', price = 890.00, version = version + 1, updated_at = SYSDATETIMEOFFSET()
WHERE sku = 'FZ-ICE-001';
```

INV-7 Deactivate a product (soft delete)

`PUT /api/v1/inventory/products/{id}/deactivate · DELETE`

Hidden from the catalogue; order history keeps it.

```
UPDATE products SET active = 0, version = version + 1, updated_at = SYSDATETIMEOFFSET()
WHERE sku = 'FZ-ICE-001';
```

INV-8 Low-stock report

`GET /api/v1/inventory/low-stock`

Quantity at or below the minimum level.

```
SELECT p.sku, p.name, i.quantity, i.minimum_level, i.maximum_level
FROM inventory i
JOIN products p ON p.id = i.product_id
WHERE i.quantity <= i.minimum_level
ORDER BY i.quantity, p.name;
```

INV-9 Set stock limits

`PUT /api/v1/inventory/products/{productId}/limits`

Maximum must be >= minimum (check constraint).

```
UPDATE i SET i.minimum_level = 20, i.maximum_level = 300, i.version = i.version + 1, i.updated_at =
    SYSDATETIMEOFFSET()
FROM inventory i JOIN products p ON p.id = i.product_id
WHERE p.sku = 'BV-COF-004';
```

INV-10 Raise a restock request

`POST /api/v1/inventory/restock-requests`

```
INSERT INTO restock_requests (product_id, requested_quantity, status, requested_by_id)
SELECT p.id, 120, 'PENDING', u.id
FROM products p CROSS JOIN app_users u
WHERE p.sku = 'DE-CHD-003' AND u.email = 'inventory@rapidmart.local';
```

INV-11 List restock requests

`GET /api/v1/inventory/restock-requests`

```
SELECT r.id, p.sku, p.name, r.requested_quantity, r.status, r.received_quantity,
       rq.email AS requested_by, ap.email AS approved_by, r.created_at
FROM restock_requests r
JOIN products p ON p.id = r.product_id
JOIN app_users rq ON rq.id = r.requested_by_id
LEFT JOIN app_users ap ON ap.id = r.approved_by_id
ORDER BY r.created_at DESC;
```

INV-12 Approve or reject a restock request

`PUT /api/v1/inventory/restock-requests/{id}/approve · /reject`

PENDING requests only. Use 'REJECTED' to reject.

```
UPDATE r
SET r.status = 'APPROVED', r.approved_by_id = (SELECT id FROM app_users WHERE email = 'inventory@rapidmart.local'),
    r.updated_at = SYSDATETIMEOFFSET()
FROM restock_requests r JOIN products p ON p.id = r.product_id
WHERE p.sku = 'BV-COF-004' AND r.status = 'PENDING';
```

INV-13 Receive stock for an approved request

`POST /api/v1/inventory/restock-requests/{id}/receive`

APPROVED only; new quantity may not exceed the maximum level. Logged as RECEIVED_STOCK.

```
SET XACT_ABORT ON;
DECLARE @Sku VARCHAR(100) = 'BV-COF-004', @Received INT = 50;
DECLARE @RequestId BIGINT, @ProductId BIGINT;
SELECT TOP 1 @RequestId = r.id, @ProductId = r.product_id
FROM restock_requests r JOIN products p ON p.id = r.product_id
WHERE p.sku = @Sku AND r.status = 'APPROVED' ORDER BY r.id;

IF @RequestId IS NULL THROW 50017, 'No approved restock request for this product.', 1;
IF (SELECT quantity + @Received - maximum_level FROM inventory WHERE product_id = @ProductId) > 0
    THROW 50018, 'Receiving this quantity would exceed the maximum stock level.', 1;

BEGIN TRANSACTION;
UPDATE inventory SET quantity = quantity + @Received, version = version + 1, updated_at = SYSDATETIMEOFFSET()
WHERE product_id = @ProductId;
UPDATE restock_requests SET status = 'RECEIVED', received_quantity = @Received, updated_at = SYSDATETIMEOFFSET()
WHERE id = @RequestId;
INSERT INTO stock_movement_history (product_id, quantity_delta, movement_type, reference_id, actor_id, actor_email)
SELECT @ProductId, @Received, 'RECEIVED_STOCK', 'RESTOCK_' + CAST(@RequestId AS VARCHAR(20)), id, email
FROM app_users WHERE email = 'inventory@rapidmart.local';
COMMIT;
```

INV-14 Stock movement history

`GET /api/v1/inventory/movements?productId=`

Newest first; filter by product.

```
DECLARE @Sku VARCHAR(100) = NULL;   -- e.g. 'DE-MLK-001'

SELECT h.created_at, p.sku, p.name, h.quantity_delta, h.movement_type, h.reference_id, h.actor_email
FROM stock_movement_history h
JOIN products p ON p.id = h.product_id
WHERE @Sku IS NULL OR p.sku = @Sku
ORDER BY h.created_at DESC, h.id DESC
OFFSET 0 ROWS FETCH NEXT 50 ROWS ONLY;
```

Delivery coordinator (8 operations)

Demo login: coordinator@rapidmart.local / DemoStaff21! (staff portal; MANAGER also has access)

Coordinators schedule deliveries to drivers, manage delivery-staff accounts, and close out failed deliveries. Demo: RM-DEMO-0003 is unassigned; driver@ (Ruwan) holds RM-DEMO-0004 and 0005; driver2@ (Chamara) is free.

ID	Operation	Endpoint
DCO-1	List deliveries (optionally by status)	`GET /api/v1/delivery-coordinator/deliveries?status=`
DCO-2	Driver availability for a date and slot	`GET /api/v1/delivery-coordinator/staff?availabilityDate=&availabilitySlot=`
DCO-3	Assign or reschedule a delivery	`POST /api/v1/delivery-coordinator/deliveries/{id}/assign`
DCO-4	Create a delivery-staff account	`POST /api/v1/delivery-coordinator/staff`
DCO-5	Update / deactivate a driver	`PUT /api/v1/delivery-coordinator/staff/{id}`
DCO-6	A driver's assignments	`GET /api/v1/delivery-coordinator/staff/{id}/assignments`
DCO-7	Mark returned items checked (failed delivery)	`POST /api/v1/delivery-coordinator/deliveries/{id}/returned-items-checked`
DCO-8	Cancel a failed delivery	`POST /api/v1/delivery-coordinator/deliveries/{id}/cancel`

DCO-1 List deliveries (optionally by status)

`GET /api/v1/delivery-coordinator/deliveries?status=`

```
DECLARE @Status VARCHAR(50) = NULL;   -- e.g. 'PENDING'

SELECT d.id, o.order_number, u.full_name AS customer, o.delivery_city, d.delivery_status,
       d.scheduled_date, d.scheduled_slot, s.full_name AS driver, p.payment_method, p.payment_status
FROM deliveries d
JOIN orders o ON o.id = d.order_id
JOIN app_users u ON u.id = o.user_id
LEFT JOIN app_users s ON s.id = d.assigned_staff_id
LEFT JOIN payments p ON p.order_id = o.id
WHERE @Status IS NULL OR d.delivery_status = @Status
ORDER BY d.scheduled_date, d.id;
```

DCO-2 Driver availability for a date and slot

`GET /api/v1/delivery-coordinator/staff?availabilityDate=&availabilitySlot=`

A driver may hold one delivery per date+slot (unique index uk_staff_slot).

```
DECLARE @Date DATE = DATEADD(DAY, 1, CAST(SYSDATETIMEOFFSET() AS DATE)), @Slot VARCHAR(50) = 'EVENING_15_18';

SELECT u.id, u.full_name, u.email, u.phone, u.active,
       CASE WHEN EXISTS (SELECT 1 FROM deliveries d WHERE d.assigned_staff_id = u.id
                         AND d.scheduled_date = @Date AND d.scheduled_slot = @Slot) THEN 0 ELSE 1 END AS available,
       (SELECT COUNT(*) FROM deliveries d WHERE d.assigned_staff_id = u.id
          AND d.delivery_status IN ('ASSIGNED', 'OUT_FOR_DELIVERY')) AS active_assignments
FROM app_users u
JOIN user_roles r ON r.user_id = u.id AND r.role = 'DELIVERY_STAFF'
ORDER BY available DESC, u.full_name;
```

DCO-3 Assign or reschedule a delivery

`POST /api/v1/delivery-coordinator/deliveries/{id}/assign`

Unpaid non-COD orders cannot be assigned; the driver must be free in that slot. Logged as a delivery attempt.

```
SET XACT_ABORT ON;
DECLARE @OrderNumber VARCHAR(100) = 'RM-DEMO-0003', @Driver VARCHAR(255) = 'driver2@rapidmart.local',
        @Date DATE = DATEADD(DAY, 1, CAST(SYSDATETIMEOFFSET() AS DATE)), @Slot VARCHAR(50) = 'EVENING_15_18';
DECLARE @DeliveryId BIGINT = (SELECT d.id FROM deliveries d JOIN orders o ON o.id = d.order_id WHERE o.order_number
    = @OrderNumber);
DECLARE @StaffId BIGINT = (SELECT u.id FROM app_users u JOIN user_roles r ON r.user_id = u.id AND r.role =
    'DELIVERY_STAFF'
                           WHERE u.email = @Driver AND u.active = 1);

IF EXISTS (SELECT 1 FROM deliveries d JOIN payments p ON p.order_id = d.order_id
           WHERE d.id = @DeliveryId AND p.payment_method <> 'COD' AND p.payment_status <> 'PAID')
    THROW 50019, 'Unpaid non-COD orders cannot be assigned for delivery.', 1;
IF EXISTS (SELECT 1 FROM deliveries WHERE assigned_staff_id = @StaffId AND scheduled_date = @Date
           AND scheduled_slot = @Slot AND id <> @DeliveryId)
```

```
    THROW 50020, 'Driver already has a delivery in this slot.', 1;

BEGIN TRANSACTION;
UPDATE deliveries SET assigned_staff_id = @StaffId, scheduled_date = @Date, scheduled_slot = @Slot,
                      delivery_status = 'ASSIGNED', updated_at = SYSDATETIMEOFFSET()
WHERE id = @DeliveryId AND delivery_status IN ('PENDING', 'ASSIGNED');
INSERT INTO delivery_attempts (delivery_id, status, notes)
SELECT @DeliveryId, 'ASSIGNED', 'Assigned to ' + full_name + ' on ' + CONVERT(VARCHAR(10), @Date, 23) + ' (' + @Slot
    + ')'
FROM app_users WHERE id = @StaffId;
COMMIT;
```

DCO-4 Create a delivery-staff account

`POST /api/v1/delivery-coordinator/staff`

Staff emails end in @rapidmart.local; the account gets DELIVERY_STAFF only. Hash below = 'DemoStaff21!'.

```
SET XACT_ABORT ON;
BEGIN TRANSACTION;
INSERT INTO app_users (email, password_hash, full_name, phone, active)
SELECT 'driver3@rapidmart.local', '$2a$10$.1fkX0pfSDayT74VaGNhZegXhSDGgEbcypkSbz78w66PPkqlEEHBi', 'Kamal Driver',
    '+94770000007', 1
WHERE NOT EXISTS (SELECT 1 FROM app_users WHERE email = 'driver3@rapidmart.local');
INSERT INTO user_roles (user_id, role)
SELECT id, 'DELIVERY_STAFF' FROM app_users u WHERE email = 'driver3@rapidmart.local'
  AND NOT EXISTS (SELECT 1 FROM user_roles r WHERE r.user_id = u.id AND r.role = 'DELIVERY_STAFF');
COMMIT;
```

DCO-5 Update / deactivate a driver

`PUT /api/v1/delivery-coordinator/staff/{id}`

Name, phone and active flag only; roles cannot change here.

```
UPDATE app_users SET full_name = 'Kamal Perera', phone = '+94770000008', active = 1, updated_at =
    SYSDATETIMEOFFSET()
WHERE email = 'driver3@rapidmart.local';
```

DCO-6 A driver's assignments

`GET /api/v1/delivery-coordinator/staff/{id}/assignments`

```
SELECT d.id, o.order_number, d.delivery_status, d.scheduled_date, d.scheduled_slot, d.completed_at
FROM deliveries d
JOIN orders o ON o.id = d.order_id
JOIN app_users s ON s.id = d.assigned_staff_id
WHERE s.email = 'driver@rapidmart.local'
ORDER BY d.scheduled_date DESC, d.id DESC;
```

DCO-7 Mark returned items checked (failed delivery)

`POST /api/v1/delivery-coordinator/deliveries/{id}/returned-items-checked`

Only FAILED deliveries.

```
DECLARE @DeliveryId BIGINT = (SELECT TOP 1 id FROM deliveries WHERE delivery_status = 'FAILED' AND
    returned_items_checked = 0);

UPDATE deliveries SET returned_items_checked = 1, updated_at = SYSDATETIMEOFFSET()
WHERE id = @DeliveryId;
INSERT INTO delivery_attempts (delivery_id, status, notes)
SELECT @DeliveryId, 'FAILED', 'Returned items checked by coordinator' WHERE @DeliveryId IS NOT NULL;
```

DCO-8 Cancel a failed delivery

`POST /api/v1/delivery-coordinator/deliveries/{id}/cancel`

Only FAILED deliveries whose returned items are checked. The order is then cancelled and restocked exactly as in CUS-14 (actor = coordinator).

```
SELECT d.id, o.order_number, d.failure_reason
FROM deliveries d JOIN orders o ON o.id = d.order_id
WHERE d.delivery_status = 'FAILED' AND d.returned_items_checked = 1;

UPDATE deliveries SET delivery_status = 'CANCELLED', updated_at = SYSDATETIMEOFFSET()
WHERE delivery_status = 'FAILED' AND returned_items_checked = 1
  AND id = (SELECT TOP 1 id FROM deliveries WHERE delivery_status = 'FAILED' AND returned_items_checked = 1);
-- then run the CUS-14 cancellation block for that order
```

Delivery staff (driver) (5 operations)

Demo login: driver@rapidmart.local, driver2@rapidmart.local / DemoStaff21! (staff portal)

Drivers see only their own deliveries, dispatch them, and record the outcome. Completing a COD delivery requires collecting exactly the order total and creates a COD settlement for the cashier. Loyalty points are earned on successful delivery. Demo: driver@ has RM-DEMO-0004 (ASSIGNED) and RM-DEMO-0005 (OUT_FOR_DELIVERY).

ID	Operation	Endpoint
DST-1	My deliveries	`GET /api/v1/delivery-staff/my-deliveries`
DST-2	My COD summary	`GET /api/v1/delivery-staff/cod-summary`
DST-3	Start a delivery (out for delivery)	`POST /api/v1/delivery-staff/deliveries/{id}/out-for-delivery`
DST-4	Complete a delivery	`POST /api/v1/delivery-staff/deliveries/{id}/complete?collectedAmount=`
DST-5	Record a failed delivery	`POST /api/v1/delivery-staff/deliveries/{id}/failed?reason=`

DST-1 My deliveries

`GET /api/v1/delivery-staff/my-deliveries`

```
SELECT d.id, o.order_number, u.full_name AS customer, o.delivery_street, o.delivery_city, o.delivery_phone,
       d.scheduled_date, d.scheduled_slot, d.delivery_status, p.payment_method, p.payment_status,
       o.total_amount AS amount_to_collect
FROM deliveries d
JOIN orders o ON o.id = d.order_id
JOIN app_users u ON u.id = o.user_id
JOIN app_users s ON s.id = d.assigned_staff_id
LEFT JOIN payments p ON p.order_id = o.id
WHERE s.email = 'driver@rapidmart.local'
ORDER BY d.scheduled_date, d.scheduled_slot;
```

DST-2 My COD summary

`GET /api/v1/delivery-staff/cod-summary`

Today's cash collected and cash not yet handed over.

```
DECLARE @DriverId BIGINT = (SELECT id FROM app_users WHERE email = 'driver@rapidmart.local');

SELECT
  (SELECT ISNULL(SUM(collected_amount), 0) FROM cod_settlements
    WHERE delivery_staff_id = @DriverId AND CAST(created_at AS DATE) = CAST(SYSDATETIMEOFFSET() AS DATE)) AS
       collected_today,
  (SELECT ISNULL(SUM(collected_amount), 0) FROM cod_settlements
    WHERE delivery_staff_id = @DriverId AND confirmed_by_cashier = 0) AS unsettled_cash,
  (SELECT COUNT(*) FROM cod_settlements WHERE delivery_staff_id = @DriverId AND confirmed_by_cashier = 0) AS
     unsettled_count;
```

DST-3 Start a delivery (out for delivery)

`POST /api/v1/delivery-staff/deliveries/{id}/out-for-delivery`

ASSIGNED -> OUT_FOR_DELIVERY; the order follows.

```
SET XACT_ABORT ON;
DECLARE @OrderNumber VARCHAR(100) = 'RM-DEMO-0004';
DECLARE @DeliveryId BIGINT = (SELECT d.id FROM deliveries d JOIN orders o ON o.id = d.order_id
                              JOIN app_users s ON s.id = d.assigned_staff_id
                              WHERE o.order_number = @OrderNumber AND s.email = 'driver@rapidmart.local'
                                AND d.delivery_status = 'ASSIGNED');
IF @DeliveryId IS NULL THROW 50021, 'Only your ASSIGNED deliveries can be dispatched.', 1;

BEGIN TRANSACTION;
UPDATE deliveries SET delivery_status = 'OUT_FOR_DELIVERY', updated_at = SYSDATETIMEOFFSET() WHERE id = @DeliveryId;
UPDATE orders SET status = 'OUT_FOR_DELIVERY', updated_at = SYSDATETIMEOFFSET() WHERE order_number = @OrderNumber;
INSERT INTO delivery_attempts (delivery_id, status, notes) VALUES (@DeliveryId, 'OUT_FOR_DELIVERY',
    'Package taken out for delivery');
COMMIT;
```

DST-4 Complete a delivery

`POST /api/v1/delivery-staff/deliveries/{id}/complete?collectedAmount=`

COD: collected amount must equal the order total; payment stays PENDING until the cashier confirms (CSH-4), and a COD settlement is created. Non-COD orders must already be PAID. Points: 1 per LKR 100 x tier multiplier.

```
SET XACT_ABORT ON;
DECLARE @OrderNumber VARCHAR(100) = 'RM-DEMO-0005', @Collected DECIMAL(12,2) = 1080.00;
DECLARE @DeliveryId BIGINT, @DriverId BIGINT, @OrderId BIGINT, @Total DECIMAL(12,2), @Method VARCHAR(50),
        @PayStatus VARCHAR(50), @CustomerId BIGINT;
SELECT @DeliveryId = d.id, @DriverId = d.assigned_staff_id, @OrderId = o.id, @Total = o.total_amount,
       @Method = p.payment_method, @PayStatus = p.payment_status, @CustomerId = o.user_id
FROM deliveries d JOIN orders o ON o.id = d.order_id JOIN payments p ON p.order_id = o.id
JOIN app_users s ON s.id = d.assigned_staff_id
WHERE o.order_number = @OrderNumber AND s.email = 'driver@rapidmart.local' AND d.delivery_status =
    'OUT_FOR_DELIVERY';

IF @DeliveryId IS NULL THROW 50022, 'Only your deliveries that are out for delivery can be completed.', 1;
IF @Method = 'COD' AND @Collected <> @Total THROW 50023, 'Collect exactly the order total for COD.', 1;
IF @Method <> 'COD' AND @PayStatus <> 'PAID' THROW 50024, 'Order payment is not completed.', 1;
BEGIN TRANSACTION;
IF @Method = 'COD'
BEGIN
    UPDATE payments SET notes = 'COD cash collected by driver@rapidmart.local; awaiting cashier verification',
                        updated_at = SYSDATETIMEOFFSET()
    WHERE order_id = @OrderId AND payment_status = 'PENDING';
    INSERT INTO cod_settlements (delivery_id, delivery_staff_id, collected_amount)
    SELECT @DeliveryId, @DriverId, @Total WHERE NOT EXISTS (SELECT 1 FROM cod_settlements WHERE delivery_id =
        @DeliveryId);
END

UPDATE deliveries SET delivery_status = 'DELIVERED', completed_at = SYSDATETIMEOFFSET(), updated_at =
    SYSDATETIMEOFFSET()
WHERE id = @DeliveryId;
UPDATE orders SET status = 'DELIVERED', updated_at = SYSDATETIMEOFFSET() WHERE id = @OrderId;
INSERT INTO delivery_attempts (delivery_id, status, notes) VALUES (@DeliveryId, 'DELIVERED',
    'Delivery completed successfully');

-- Loyalty points, exactly once per order
IF NOT EXISTS (SELECT 1 FROM reward_transactions WHERE reference_id = 'ORDER_' + @OrderNumber AND reason =
    'EARNED_ORDER_COMPLETED')
BEGIN
    IF NOT EXISTS (SELECT 1 FROM loyalty_accounts WHERE user_id = @CustomerId)
        INSERT INTO loyalty_accounts (user_id) VALUES (@CustomerId);
    DECLARE @AccountId BIGINT, @Multiplier DECIMAL(5,2);
    SELECT @AccountId = la.id, @Multiplier = ISNULL(t.earning_rate_multiplier, 1.00)
    FROM loyalty_accounts la LEFT JOIN loyalty_tiers t ON t.id = la.current_tier_id WHERE la.user_id = @CustomerId;
    DECLARE @Points INT = FLOOR(@Total * 0.01 * @Multiplier);
    IF @Points > 0
    BEGIN
        UPDATE loyalty_accounts SET current_points = current_points + @Points, lifetime_points = lifetime_points +
            @Points,
                                    version = version + 1, updated_at = SYSDATETIMEOFFSET() WHERE id = @AccountId;
        UPDATE la SET la.current_tier_id = (SELECT TOP 1 t.id FROM loyalty_tiers t
                                            WHERE t.minimum_points <= la.lifetime_points ORDER BY t.minimum_points
                                                DESC)
        FROM loyalty_accounts la WHERE la.id = @AccountId;
        INSERT INTO reward_transactions (loyalty_account_id, points_delta, balance_after, reason, reference_id,
            actor_id)
        SELECT @AccountId, @Points, current_points, 'EARNED_ORDER_COMPLETED', 'ORDER_' + @OrderNumber, @CustomerId
        FROM loyalty_accounts WHERE id = @AccountId;
    END
END
COMMIT;
```

DST-5 Record a failed delivery

`POST /api/v1/delivery-staff/deliveries/{id}/failed?reason=`

A reason is required. The coordinator then checks returned items (DCO-7/8).

```
SET XACT_ABORT ON;
DECLARE @OrderNumber VARCHAR(100) = 'RM-DEMO-0004', @Reason VARCHAR(MAX) =
    'Customer not at home; phone unreachable';
DECLARE @DeliveryId BIGINT = (SELECT d.id FROM deliveries d JOIN orders o ON o.id = d.order_id
                              JOIN app_users s ON s.id = d.assigned_staff_id
                              WHERE o.order_number = @OrderNumber AND s.email = 'driver@rapidmart.local'
                                AND d.delivery_status IN ('ASSIGNED', 'OUT_FOR_DELIVERY'));
IF @DeliveryId IS NULL THROW 50025, 'No open delivery of yours for this order.', 1;

BEGIN TRANSACTION;
UPDATE deliveries SET delivery_status = 'FAILED', failure_reason = @Reason, updated_at = SYSDATETIMEOFFSET()
WHERE id = @DeliveryId;
INSERT INTO delivery_attempts (delivery_id, status, notes) VALUES (@DeliveryId, 'FAILED', @Reason);
COMMIT;
```

Loyalty officer (8 operations)

Demo login: loyalty@rapidmart.local / DemoStaff21! (staff portal; MANAGER also has access)

Loyalty officers manage tiers and offers and make audited manual point adjustments. Tiers and offers use optimistic locking: an update must supply the version it read. Demo: BRONZE/SILVER/GOLD/PLATINUM tiers; offers WELCOME5, FRESH10 (Silver+), GOLD15 (Gold+).

ID	Operation	Endpoint
LOY-1	List tiers	`GET /api/v1/loyalty-officer/tiers`
LOY-2	Create a tier	`POST /api/v1/loyalty-officer/tiers`
LOY-3	Edit a tier (optimistic lock)	`PUT /api/v1/loyalty-officer/tiers/{id}`
LOY-4	List offers	`GET /api/v1/loyalty-officer/offers`
LOY-5	Create an offer	`POST /api/v1/loyalty-officer/offers`
LOY-6	Edit / deactivate an offer (optimistic lock)	`PUT /api/v1/loyalty-officer/offers/{id}`
LOY-7	Adjust a customer's points	`POST /api/v1/loyalty-officer/adjust-points`
LOY-8	Manual adjustment history	`GET /api/v1/loyalty-officer/adjustments`

LOY-1 List tiers

`GET /api/v1/loyalty-officer/tiers`

```
SELECT id, tier_name, minimum_points, earning_rate_multiplier, benefits_description, version
FROM loyalty_tiers
ORDER BY minimum_points;
```

LOY-2 Create a tier

`POST /api/v1/loyalty-officer/tiers`

Tier names are unique.

```
INSERT INTO loyalty_tiers (tier_name, minimum_points, earning_rate_multiplier, benefits_description)
SELECT 'DIAMOND', 10000, 2.50, 'Earn 2.5x points and a dedicated shopper'
WHERE NOT EXISTS (SELECT 1 FROM loyalty_tiers WHERE tier_name = 'DIAMOND');
```

LOY-3 Edit a tier (optimistic lock)

`PUT /api/v1/loyalty-officer/tiers/{id}`

0 rows updated = someone else changed it first (the app returns HTTP 409).

```
DECLARE @ReadVersion BIGINT = (SELECT version FROM loyalty_tiers WHERE tier_name = 'DIAMOND');

UPDATE loyalty_tiers
SET minimum_points = 8000, earning_rate_multiplier = 2.50, version = version + 1
WHERE tier_name = 'DIAMOND' AND version = @ReadVersion;

IF @@ROWCOUNT = 0 PRINT 'Stale version - reload and try again.';
```

LOY-4 List offers

`GET /api/v1/loyalty-officer/offers`

```
SELECT o.id, o.code, o.title, o.discount_percentage, o.min_order_amount, o.start_date, o.end_date,
       t.tier_name AS eligible_tier, o.active, o.version
FROM offers o
LEFT JOIN loyalty_tiers t ON t.id = o.eligible_tier_id
ORDER BY o.start_date DESC, o.code;
```

LOY-5 Create an offer

`POST /api/v1/loyalty-officer/offers`

Code stored upper-case and unique; 0 < discount <= 100; end date >= start date.

```
INSERT INTO offers (title, code, discount_percentage, min_order_amount, start_date, end_date, eligible_tier_id,
    active)
SELECT 'Platinum Weekend 20% Off', 'PLAT20', 20.00, 4000.00,
       CAST(SYSDATETIMEOFFSET() AS DATE), DATEADD(MONTH, 1, CAST(SYSDATETIMEOFFSET() AS DATE)),
       (SELECT id FROM loyalty_tiers WHERE tier_name = 'PLATINUM'), 1
WHERE NOT EXISTS (SELECT 1 FROM offers WHERE code = 'PLAT20');
```

LOY-6 Edit / deactivate an offer (optimistic lock)

`PUT /api/v1/loyalty-officer/offers/{id}`

```
DECLARE @ReadVersion BIGINT = (SELECT version FROM offers WHERE code = 'PLAT20');

UPDATE offers
SET discount_percentage = 18.00, active = 1, version = version + 1, updated_at = SYSDATETIMEOFFSET()
WHERE code = 'PLAT20' AND version = @ReadVersion;
```

LOY-7 Adjust a customer's points

`POST /api/v1/loyalty-officer/adjust-points`

Customers only; delta != 0; the balance may never go negative. Positive adjustments also raise lifetime points and can move the tier. Recorded as 'MANUAL_ADJUSTMENT: <reason>' and audited.

```
SET XACT_ABORT ON;
DECLARE @Customer VARCHAR(255) = 'customer1@example.com', @Delta INT = 25, @Reason VARCHAR(200) =
    'Late delivery goodwill';
DECLARE @UserId BIGINT = (SELECT u.id FROM app_users u JOIN user_roles r ON r.user_id = u.id AND r.role = 'CUSTOMER'
                          WHERE u.email = @Customer);
DECLARE @OfficerId BIGINT = (SELECT id FROM app_users WHERE email = 'loyalty@rapidmart.local');
IF @UserId IS NULL THROW 50026, 'Reward points can only be adjusted for customer accounts.', 1;
IF @Delta = 0 THROW 50027, 'Points delta cannot be zero.', 1;

BEGIN TRANSACTION;
IF NOT EXISTS (SELECT 1 FROM loyalty_accounts WHERE user_id = @UserId)
    INSERT INTO loyalty_accounts (user_id) VALUES (@UserId);
IF (SELECT current_points + @Delta FROM loyalty_accounts WHERE user_id = @UserId) < 0
    THROW 50028, 'Adjustment would make the balance negative.', 1;
UPDATE loyalty_accounts
SET current_points = current_points + @Delta,
    lifetime_points = lifetime_points + CASE WHEN @Delta > 0 THEN @Delta ELSE 0 END,
    version = version + 1, updated_at = SYSDATETIMEOFFSET()
WHERE user_id = @UserId;
IF @Delta > 0
    UPDATE la SET la.current_tier_id = (SELECT TOP 1 t.id FROM loyalty_tiers t
                                        WHERE t.minimum_points <= la.lifetime_points ORDER BY t.minimum_points DESC)
    FROM loyalty_accounts la WHERE la.user_id = @UserId;

INSERT INTO reward_transactions (loyalty_account_id, points_delta, balance_after, reason, reference_id, actor_id)
SELECT id, @Delta, current_points, 'MANUAL_ADJUSTMENT: ' + @Reason, NULL, @OfficerId
FROM loyalty_accounts WHERE user_id = @UserId;
INSERT INTO audit_logs (action, actor_email, target_reference, details)
VALUES ('MANUAL_REWARD_ADJUSTMENT', 'loyalty@rapidmart.local', @Customer,
        'Adjusted points by ' + CAST(@Delta AS VARCHAR(10)) + ' for reason: ' + @Reason);
COMMIT;
```

LOY-8 Manual adjustment history

`GET /api/v1/loyalty-officer/adjustments`

```
SELECT t.created_at, u.full_name AS customer, u.email, t.points_delta, t.balance_after,
       LTRIM(SUBSTRING(t.reason, LEN('MANUAL_ADJUSTMENT:') + 1, 255)) AS reason, a.email AS actor
FROM reward_transactions t
JOIN loyalty_accounts la ON la.id = t.loyalty_account_id
JOIN app_users u ON u.id = la.user_id
LEFT JOIN app_users a ON a.id = t.actor_id
WHERE t.reason LIKE 'MANUAL[_]ADJUSTMENT:%'
ORDER BY t.created_at DESC;
```

Manager (14 operations)

Demo login: manager@rapidmart.local / DemoManager21! (staff portal)

Managers administer employees, see reports and the audit trail, answer feedback, and manage their own profile. They can also use every cashier, inventory, coordinator and loyalty operation above, but they are staff only (no cart or orders). Demo: a pending email change to store.manager@rapidmart.local.

ID	Operation	Endpoint
MGR-1	View my profile	`GET /api/v1/manager/profile`
MGR-2	Update my profile	`PUT /api/v1/manager/profile`
MGR-3	Request an email change	`POST /api/v1/manager/profile/email-change`
MGR-4	Change my password	`POST /api/v1/manager/profile/password`
MGR-5	List employees	`GET /api/v1/manager/employees`
MGR-6	Create an employee	`POST /api/v1/manager/employees`
MGR-7	Update an employee's roles / deactivate	`PUT /api/v1/manager/employees/{id}`
MGR-8	Sales report	`GET /api/v1/manager/reports/sales?startDate=&endDate=`
MGR-9	Sales CSV export (rows)	`GET /api/v1/manager/reports/sales/csv`
MGR-10	Inventory and delivery summaries	`GET /api/v1/manager/reports/inventory · /reports/delivery`
MGR-11	Audit log	`GET /api/v1/manager/audit-logs`
MGR-12	Feedback inbox	`GET /api/v1/manager/feedback`
MGR-13	Reply to feedback	`POST /api/v1/manager/feedback/{id}/reply`
MGR-14	Close feedback	`POST /api/v1/manager/feedback/{id}/close`

MGR-1 View my profile

`GET /api/v1/manager/profile`

```
SELECT id, email, full_name, phone, active, last_login_at, created_at
FROM app_users WHERE email = 'manager@rapidmart.local';
```

MGR-2 Update my profile

`PUT /api/v1/manager/profile`

Email cannot be changed here. Audited.

```
UPDATE app_users SET full_name = 'System Manager', phone = '+94770000009', updated_at = SYSDATETIMEOFFSET()
WHERE email = 'manager@rapidmart.local';
INSERT INTO audit_logs (action, actor_email, target_reference, details)
VALUES ('MANAGER_PROFILE_UPDATED', 'manager@rapidmart.local', 'manager@rapidmart.local', 'Updated name and phone');
```

MGR-3 Request an email change

`POST /api/v1/manager/profile/email-change`

Replaces any earlier PENDING request; the login email stays the same until approved.

```
SET XACT_ABORT ON;
DECLARE @UserId BIGINT = (SELECT id FROM app_users WHERE email = 'manager@rapidmart.local');
DECLARE @NewEmail VARCHAR(255) = 'ops.manager@rapidmart.local';
IF EXISTS (SELECT 1 FROM app_users WHERE email = @NewEmail) THROW 50029, 'Email is already registered.', 1;

BEGIN TRANSACTION;
UPDATE email_change_requests SET status = 'CANCELLED', updated_at = SYSDATETIMEOFFSET()
WHERE user_id = @UserId AND status = 'PENDING';
INSERT INTO email_change_requests (user_id, new_email, status) VALUES (@UserId, @NewEmail, 'PENDING');
INSERT INTO audit_logs (action, actor_email, target_reference, details)
VALUES ('EMAIL_CHANGE_REQUESTED', 'manager@rapidmart.local', 'manager@rapidmart.local', 'Requested change to ' +
    @NewEmail);
COMMIT;
SELECT new_email, status, created_at FROM email_change_requests WHERE user_id = @UserId ORDER BY created_at DESC;
```

MGR-4 Change my password

`POST /api/v1/manager/profile/password`

BCrypt hashing happens in the app; the database stores the new hash and the change is audited.

```
-- @NewHash must be a BCrypt hash produced by the application.
DECLARE @NewHash VARCHAR(255) = '$2a$10$hwLZxk3WaZG.Z1sfaPNHjOB/PYGb9ewg4KOxPNS99tU.RdAiawfGO';
UPDATE app_users SET password_hash = @NewHash, updated_at = SYSDATETIMEOFFSET()
WHERE email = 'manager@rapidmart.local';
INSERT INTO audit_logs (action, actor_email, target_reference, details)
VALUES ('PASSWORD_CHANGED', 'manager@rapidmart.local', 'manager@rapidmart.local', 'Manager changed their password');
```

MGR-5 List employees

`GET /api/v1/manager/employees`

Users with at least one staff role.

```
SELECT u.id, u.email, u.full_name, u.phone, u.active, u.last_login_at, STRING_AGG(r.role, ', ') AS roles
FROM app_users u
JOIN user_roles r ON r.user_id = u.id
WHERE EXISTS (SELECT 1 FROM user_roles s WHERE s.user_id = u.id AND s.role <> 'CUSTOMER')
GROUP BY u.id, u.email, u.full_name, u.phone, u.active, u.last_login_at
ORDER BY u.full_name;
```

MGR-6 Create an employee

`POST /api/v1/manager/employees`

Staff email @rapidmart.local, one or more staff roles. Hash below = 'DemoStaff21!'. Audited.

```
SET XACT_ABORT ON;
BEGIN TRANSACTION;
INSERT INTO app_users (email, password_hash, full_name, phone, active)
VALUES ('cashier2@rapidmart.local', '$2a$10$.1fkX0pfSDayT74VaGNhZegXhSDGgEbcypkSbz78w66PPkqlEEHBi',
    'Tharindu Cashier', '+94770000010', 1);
INSERT INTO user_roles (user_id, role) VALUES (SCOPE_IDENTITY(), 'CASHIER');
INSERT INTO audit_logs (action, actor_email, target_reference, details)
VALUES ('EMPLOYEE_CREATED', 'manager@rapidmart.local', 'cashier2@rapidmart.local',
    'Created employee with role CASHIER');
COMMIT;
```

MGR-7 Update an employee's roles / deactivate

`PUT /api/v1/manager/employees/{id}`

Roles are replaced as a set. A manager cannot remove their own MANAGER role or deactivate themselves.

```
SET XACT_ABORT ON;
DECLARE @UserId BIGINT = (SELECT id FROM app_users WHERE email = 'cashier2@rapidmart.local');
BEGIN TRANSACTION;
UPDATE app_users SET full_name = 'Tharindu Silva', active = 1, updated_at = SYSDATETIMEOFFSET() WHERE id = @UserId;
DELETE FROM user_roles WHERE user_id = @UserId;
INSERT INTO user_roles (user_id, role) VALUES (@UserId, 'CASHIER'), (@UserId, 'INVENTORY_STAFF');
COMMIT;
```

MGR-8 Sales report

`GET /api/v1/manager/reports/sales?startDate=&endDate=`

Order-creation dates inclusive; excludes CANCELLED and AWAITING_PAYMENT orders.

```
DECLARE @Start DATE = DATEADD(DAY, -30, CAST(SYSDATETIMEOFFSET() AS DATE)), @End DATE = CAST(SYSDATETIMEOFFSET() AS
    DATE);

WITH sales AS (
    SELECT o.id, o.total_amount FROM orders o
    WHERE CAST(o.created_at AS DATE) BETWEEN @Start AND @End
      AND o.status NOT IN ('CANCELLED', 'AWAITING_PAYMENT'))
SELECT COUNT(*) AS total_orders,
       ISNULL(SUM(s.total_amount), 0) AS total_revenue,
       ISNULL((SELECT SUM(r.refunded_amount) FROM refund_requests r
               WHERE r.refund_status = 'REFUNDED' AND r.order_id IN (SELECT id FROM sales)), 0) AS refunded_total,
       ISNULL(SUM(s.total_amount), 0)
         - ISNULL((SELECT SUM(r.refunded_amount) FROM refund_requests r
                   WHERE r.refund_status = 'REFUNDED' AND r.order_id IN (SELECT id FROM sales)), 0) AS net_revenue
FROM sales s;
```

MGR-9 Sales CSV export (rows)

`GET /api/v1/manager/reports/sales/csv`

One row per order in the range.

```
SELECT o.order_number AS [Order Number], u.email AS [Customer Email], o.delivery_date AS [Delivery Date],
       o.status AS [Status], o.total_amount AS [Total Amount (LKR)], CAST(o.created_at AS DATE) AS [Order Date],
       ISNULL(r.refund_amount, 0) AS [Refunded Amount (LKR)]
FROM orders o
JOIN app_users u ON u.id = o.user_id
LEFT JOIN refund_requests r ON r.order_id = o.id AND r.refund_status = 'REFUNDED'
ORDER BY o.created_at;
```

MGR-10 Inventory and delivery summaries

`GET /api/v1/manager/reports/inventory · /reports/delivery`

```
SELECT (SELECT COUNT(*) FROM inventory) AS total_products,
       (SELECT COUNT(*) FROM inventory WHERE quantity <= minimum_level) AS low_stock_count;

SELECT delivery_status, COUNT(*) AS deliveries
FROM deliveries
GROUP BY delivery_status
ORDER BY deliveries DESC;
```

MGR-11 Audit log

`GET /api/v1/manager/audit-logs`

Newest first, paged.

```
SELECT [timestamp], action, actor_email, target_reference, details
FROM audit_logs
ORDER BY [timestamp] DESC
OFFSET 0 ROWS FETCH NEXT 50 ROWS ONLY;
```

MGR-12 Feedback inbox

`GET /api/v1/manager/feedback`

```
SELECT f.id, u.full_name AS customer, u.email, f.category, f.subject, f.message, f.status, f.created_at,
       (SELECT COUNT(*) FROM feedback_replies r WHERE r.feedback_id = f.id) AS replies
FROM feedbacks f
JOIN app_users u ON u.id = f.user_id
ORDER BY CASE f.status WHEN 'PENDING' THEN 0 WHEN 'RESPONDED' THEN 1 ELSE 2 END, f.created_at DESC;
```

MGR-13 Reply to feedback

`POST /api/v1/manager/feedback/{id}/reply`

Not allowed once CLOSED; marks it RESPONDED.

```
SET XACT_ABORT ON;
DECLARE @FeedbackId BIGINT = (SELECT TOP 1 f.id FROM feedbacks f JOIN app_users u ON u.id = f.user_id
                              WHERE u.email = 'customer1@example.com' AND f.status <> 'CLOSED' ORDER BY f.id);
IF @FeedbackId IS NULL THROW 50030, 'Feedback not found or already closed.', 1;

BEGIN TRANSACTION;
INSERT INTO feedback_replies (feedback_id, replied_by_id, reply_message)
SELECT @FeedbackId, id, 'Thank you! We have shared your comments with the delivery team.'
FROM app_users WHERE email = 'manager@rapidmart.local';
UPDATE feedbacks SET status = 'RESPONDED', updated_at = SYSDATETIMEOFFSET() WHERE id = @FeedbackId;
COMMIT;
```

MGR-14 Close feedback

`POST /api/v1/manager/feedback/{id}/close`

Idempotent.

```
UPDATE f SET f.status = 'CLOSED', f.updated_at = SYSDATETIMEOFFSET()
FROM feedbacks f
WHERE f.subject = 'Bananas were overripe' AND f.status <> 'CLOSED';
```